Stack

Definition

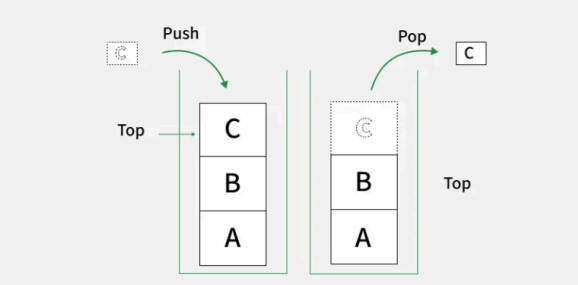
A **Stack** is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**

LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.

It behaves like a stack of plates, where the last plate added is the first one to be removed. Think of it this way:

Pushing an element onto the stack is like adding a new plate on top.

Popping an element removes the top plate from the stack.



LIFO(Last In First Out) Principle

The LIFO principle means that the last element added to a stack is the first one to be removed.

- New elements are always pushed on top.
- Removal (pop) also happens only from the top.
- This ensures a strict order: last in → first out.

Real-world examples of LIFO:

- **Stack of plates** – The last plate placed on top is the first one you pick up.

- **Stack of books** – Books are added and removed from the top, so the last book placed is the first one taken.

Basic Terminologies of Stack

- **Top**: The position of the most recently inserted element. Insertions (push) and deletions (pop) are always performed at the top.
- **Size**: Refers to the current number of elements present in the stack.

Types of Stack:

Fixed Size Stack

- A fixed size stack has a predefined capacity.
- Once it becomes full, no more elements can be added (this causes **overflow**).
- If the stack is empty and we try to remove an element, it causes **underflow**.
- Typically implemented using a static array.

Example: Declaring a stack of size 10 using an array.

Dynamic Size Stack

- A dynamic size stack can grow and shrink automatically as needed.
- If the stack is full, its capacity expands to allow more elements.
- As elements are removed, memory usage can shrink as well.
- Can be implemented using:> **Linked List** → grows/shrinks naturally.> **Dynamic Array** (like vector in C++ or Array List in Java) → resizes automatically.

Example: Stack implementation using linked list or resizable array.

Note: We generally use dynamic stacks in practice, as they can grow or shrink as needed without overflow issues.

Fixed Size Stack Vs Dynamic Size Stack

Features	Fixed Size	Dynamic Size
Memory Allocation	Fixed at the time of stack creation	Grows or shrinks automatically as elements change
Flexibility	Limited, cannot exceed predefined capacity	Highly flexible, no predefined size limit
Performance	Slightly faster due to no resizing overhead	May have resizing cost (amortized O(1) push)
Ease of Use	Need to handle overflow/ underflow manually	More convenient, no overflow (only underflow)
Use Case	Useful when maximum size is known in advance	Useful when size requirements are unknown or variable

Common Operations on Stack:

In order to make manipulations in a stack, there are certain operations provided to us.

- **push()** to insert an element into the stack.
- **pop()** to remove an element from the stack.
- **peek()** Returns the top element of the stack.
- **isEmpty()** returns true if stack is empty else false.
- **size()** returns the size of the stack.

Implementation using Array

A stack is a **linear data structure** that follows the **Last-In-First-Out (LIFO)** principle. It can be implemented using an array by treating the end of the array as the top of the stack.

A stack can be implemented using an array where we maintain:

- An integer array to store elements.
- A variable capacity to represent the maximum size of the stack.

- A variable top to track the index of the top element. Initially, top = -1 to indicate an empty stack.

```
class myStack {

    // array to store elements
    int *arr;

    // maximum size of stack
    int capacity;

    // index of top element
    int top;

public:

    // constructor
    myStack(int cap) {
        capacity = cap;
        arr = new int[capacity];
        top = -1;
    }
};
```

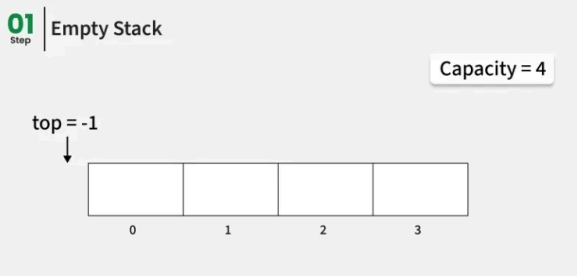
Operations On Stack

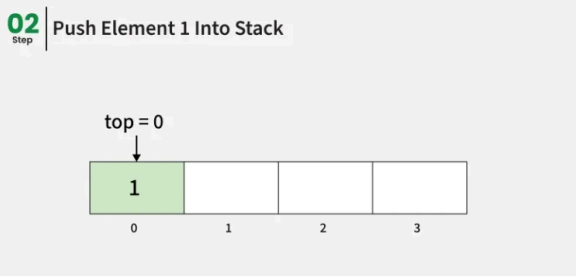
Push Operation:

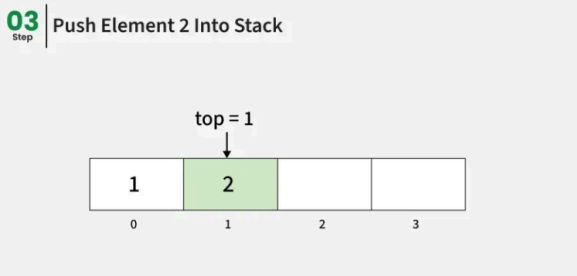
Adds an item to the stack. If the stack is full, then it is said to be an **Overflow** condition.

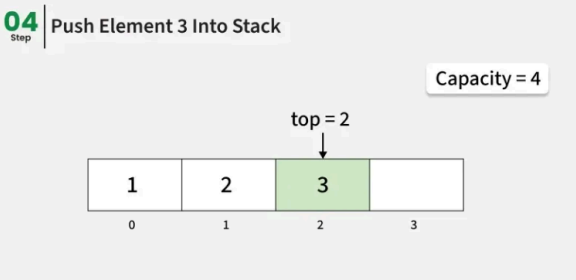
- Before pushing the element to the stack, we check if the stack is full.
- If the stack is full (top == capacity-1) , then Stack Overflows and we cannot insert the element to the stack.

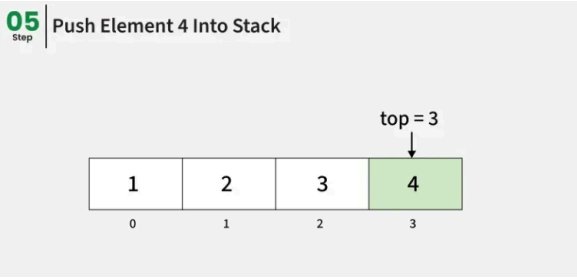
- Otherwise, we increment the value of top by 1 (top = top + 1) and the new value is inserted at top position .
- The elements can be pushed into the stack till we reach the capacity of the stack.

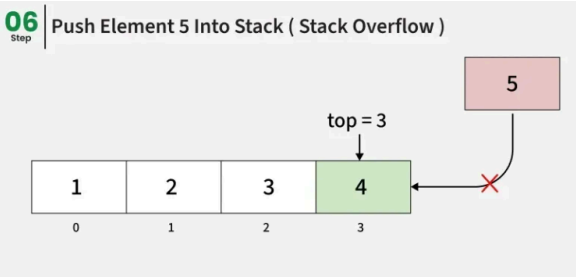













```
void push(int x) {

    if (top == capacity - 1) {
        cout << "Stack Overflow\n";
        return;
    }
```

```
    arr[++top] = x;
}
```

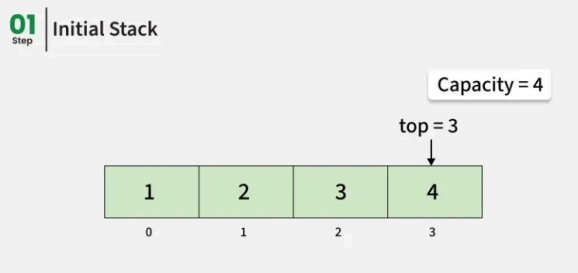
Time Complexity: O(1)

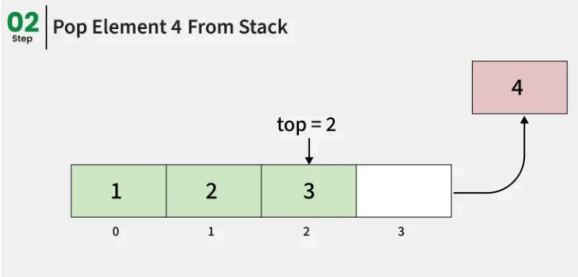
Auxiliary Space: O(1)

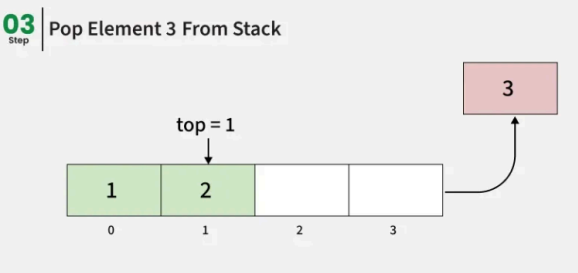
Pop Operation:

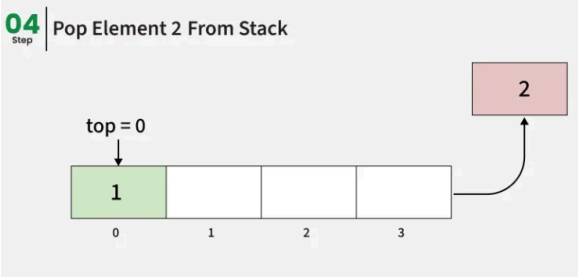
Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow** condition.

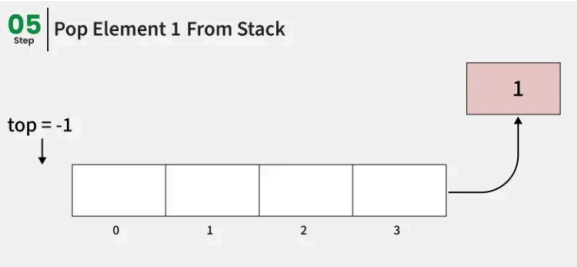
- Before popping the element from the stack, we check if the stack is empty .
- If the stack is empty (top == -1), then Stack Underflows and we cannot remove any element from the stack.
- Otherwise, we store the value at top, decrement the value of top by 1 (top = top – 1) and return the stored top value.

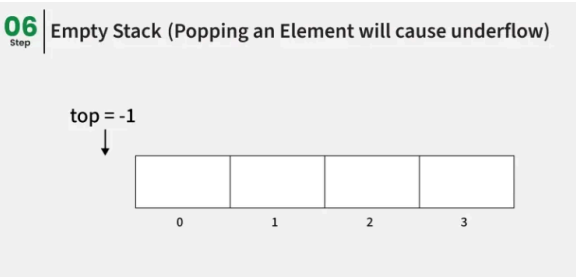













```
int pop() {

    if (top == -1) {
        cout << "Stack Underflow\n";
        return -1;
    }

    return arr[top--];
}
```

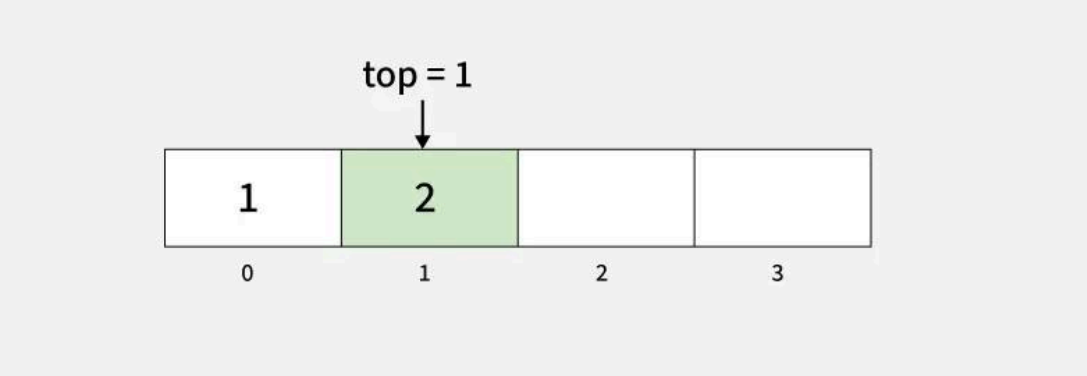
Time Complexity: O(1)

Auxiliary Space: O(1)

Top or Peek Operation in Stack:

Returns the top element of the stack.

- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty (top == -1), we simply print “Stack is empty”.
- Otherwise, we return the element stored at **index = top**.




```
int peek() {

    if (top == -1) {
        cout << "Stack is Empty\n";
        return -1;
    }

    return arr[top];
}
```

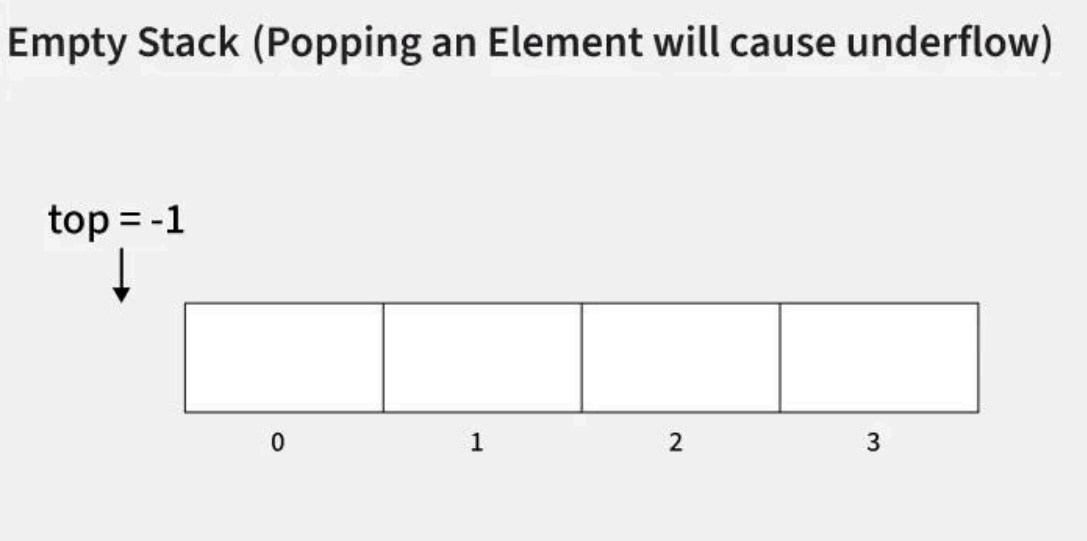
Time Complexity: O(1)

Auxiliary Space: O(1)

isEmpty Operation in Stack:

Returns true if the stack is empty, else false.

- Check for the value of top in stack.
- If (top == -1) , then the stack is empty so return true.
- Otherwise, the stack is not empty so return false.




```
bool isEmpty() {

    return top == -1;
}
```

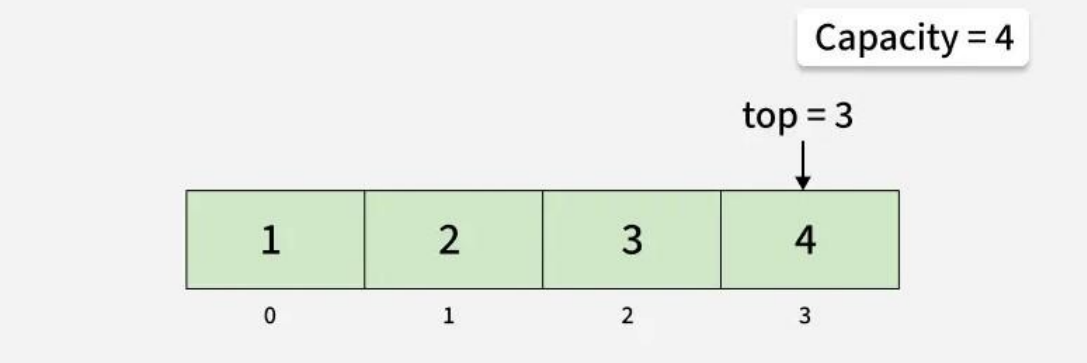
Time Complexity: O(1)

Auxiliary Space: O(1)

isFull Operation in Stack :

Returns true if the stack is full, else false.

- Check for the value of top in stack.
- If (top == capacity-1), then the stack is full so return true.
- Otherwise, the stack is not full so return false.



Time Complexity: O(1)

Auxiliary Space: O(1)

Full Implementation of Stack using Array

```
#include <iostream>
using namespace std;

class myStack {

    // array to store elements
    int *arr;

    // maximum size of stack
    int capacity;

    // index of top element
    int top;

public:

    // constructor
    myStack(int cap) {
        capacity = cap;
        arr = new int[capacity];
        top = -1;
    }
```

```
// push operation
void push(int x) {
    if (top == capacity - 1) {
        cout << "Stack Overflow\n";
        return;
    }
    arr[++top] = x;
}

// pop operation
int pop() {
    if (top == -1) {
        cout << "Stack Underflow\n";
        return -1;
    }
    return arr[top--];
}

// peek (or top) operation
int peek() {
    if (top == -1) {
        cout << "Stack is Empty\n";
        return -1;
    }
    return arr[top];
}

// check if stack is empty
bool isEmpty() {
    return top == -1;
}

// check if stack is full
bool isFull() {
    return top == capacity - 1;
```

```
    }
};

int main() {
    myStack st(4);

    // pushing elements
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);

    // popping one element
    cout << "Popped: " << st.pop() << "\n";

    // checking top element
    cout << "Top element: " << st.peek() << "\n";

    // checking if stack is empty
    cout << "Is stack empty: " << (st.isEmpty() ? "Yes" : "N
o") << "\n";

    // checking if stack is full
    cout << "Is stack full: " << (st.isFull() ? "Yes" : "No")
<< "\n";

    return 0;
}
```

💻

Popped : 4
Top element : 4
Is stack empty : No
Is stack full : No

Stack Implementation using Dynamic Array(Vector)

```cpp
#include <iostream>
#include <vector>
using namespace std;

class myStack {
    vector<int> arr;

public:

    // push operation
    void push(int x) {
        arr.push_back(x);
    }

    // pop operation
    int pop() {
        if (arr.empty()) {
            cout << "Stack Underflow" << endl;
            return -1;
        }
        int val = arr.back();
        arr.pop_back();
        return val;
    }

    // peek operation
    int peek() {
        if (arr.empty()) {
            cout << "Stack is Empty" << endl;
            return -1;
        }
        return arr.back();
    }
```

```
    // check if stack is empty
    bool isEmpty() {
        return arr.empty();
    }

    // current size
    int size() {
        return arr.size();
    }
};

int main() {
    myStack st;

    // pushing elements
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);

    // popping one element
    cout << "Popped: " << st.pop() << endl;

    // checking top element
    cout << "Top element: " << st.peek() << endl;

    // checking if stack is empty
    cout << "Is stack empty: " << (st.isEmpty() ? "Yes" : "N
o") << endl;

    // checking current size
    cout << "Current size: " << st.size() << endl;
}
```

💻 Popped : 4
Top element : 3
Is stack empty : No
Current size : 3

Fixed Size Stack Vs Dynamic Size Stack

Features	Fixed Size	Dynamic Size
Memory Allocation	Fixed at the time of stack creation	Grows or shrinks automatically as elements change
Flexibility	Limited, cannot exceed predefined capacity	Highly flexible, no predefined size limit
Performance	Slightly faster due to no resizing overhead	May have resizing cost (amortized O(1) push)
Ease of Use	Need to handle overflow/ underflow manually	More convenient, no overflow (only underflow)
Use Case	Useful when maximum size is known in advance	Useful when size requirements are unknown or variable

Implementation using Linked List

A stack is a linear data structure that follows the **Last-In-First-Out (LIFO)** principle. It can be implemented using a **linked list**, where each element of the stack is represented as a node. The head of the linked list acts as the top of the stack.

Declaration of Stack using Linked List

A stack can be implemented using a **linked list** where we maintain:

- A Node structure/class that contains:data → to store the element.next → pointer/reference to the next node in the stack.
- A pointer/reference top that always points to the current **top node** of the stack.Initially, top = null to represent an empty stack.

```
// Node structure
class Node {
public:
    int data;
    Node* next;

    Node(int x) {
        data = x;
        next = NULL;
    }
};

// Stack class
class myStack {

    // pointer to top node
    Node* top;

public:
    myStack() {

        // initially stack is empty
        top = NULL;
    }
};
```

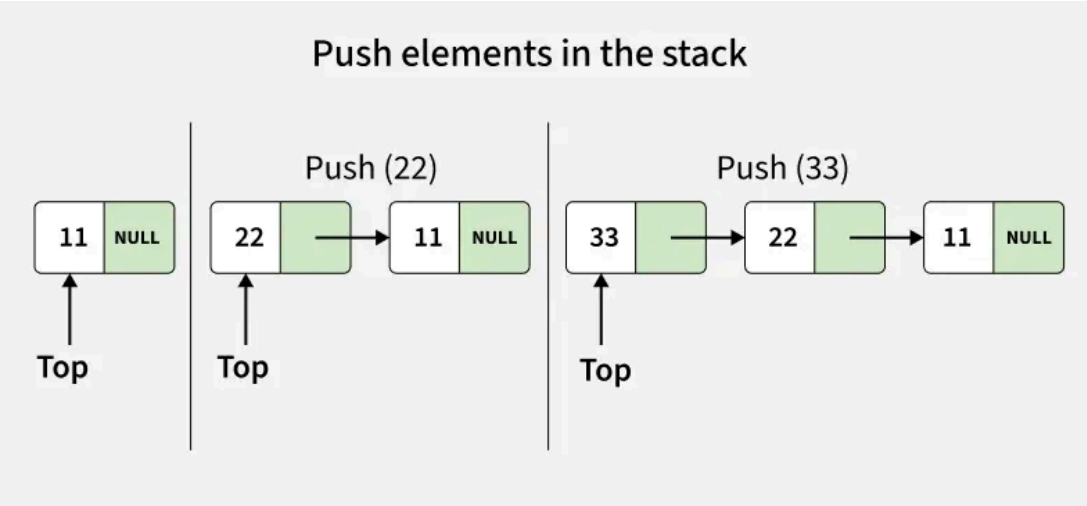
Operations on Stack using Linked List

Push Operation

Adds an item to the stack. Unlike array implementation, there is no fixed capacity in linked list. Overflow occurs only when memory is exhausted.

- A new node is created with the given value.

- The new node’s next pointer is set to the current top.
- The top pointer is updated to point to this new node.




```
void push(int x) {
    Node* temp = new Node(x);
    temp->next = top;
    top = temp;
}
```

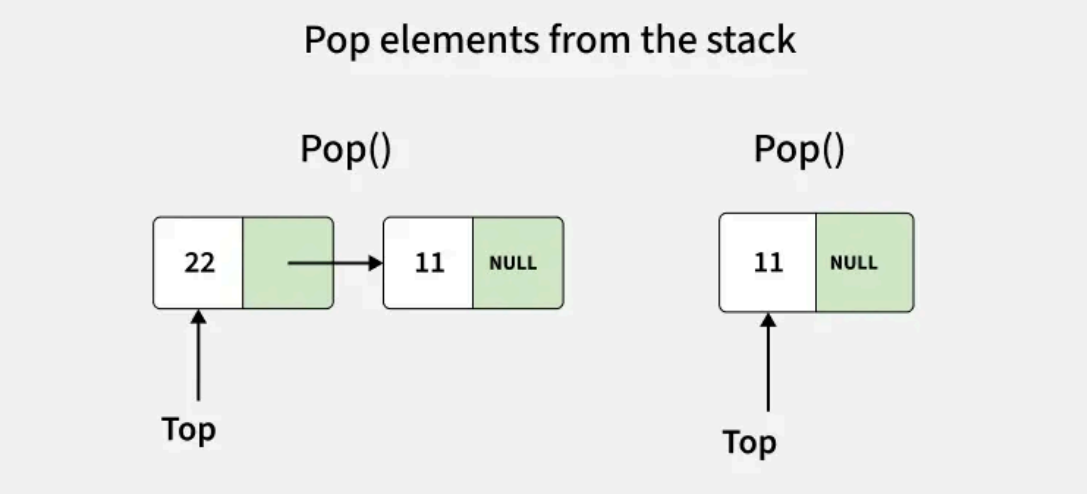
Time Complexity: O(1)

Auxiliary Space: O(1)

Pop Operation

Removes the top item from the stack. If the stack is empty, it is said to be an **Underflow condition**.

- Before deleting, we check if the stack is empty (top == NULL).
- If the stack is empty, underflow occurs and deletion is not possible.
- Otherwise, we store the current top node in a temporary pointer.
- Move the top pointer to the next node.
- Delete the temporary node to free memory.




```
int pop() {

    if (top == NULL) {
        cout << "Stack Underflow" << endl;
        return -1;
    }

    Node* temp = top;
    top = top->next;
    int val = temp->data;

    delete temp;
    return val;
}
```

Time Complexity: O(1)

Auxiliary Space: O(1)

Peek (or Top) Operation

Returns the value of the top item without removing it from the stack.

- If the stack is empty (top == NULL), then no element exists.
- Otherwise, simply return the data of the node pointed by top.

```
int peek() {

    if (top == NULL) {
        cout << "Stack is Empty" << endl;
        return -1;
    }

    return top->data;
}
```

Time Complexity: O(1)

Auxiliary Space: O(1)

isEmpty Operation

Checks whether the stack has no elements.

- If the top pointer is NULL, it means the stack is empty and the function returns true.
- Otherwise, it returns false.

```
bool isEmpty() {
    return top == NULL;
}
```

Time Complexity: O(1)

Auxiliary Space: O(1)

Stack Implementation using Linked List

```
#include <iostream>
using namespace std;

// Node structure
class Node {
public:
```

```
    int data;
    Node* next;

    Node(int x) {
        data = x;
        next = NULL;
    }
};

// Stack implementation using linked list
class myStack {
    Node* top;

    // To Store current size of stack
    int count;

public:
    myStack() {

        // initially stack is empty
        top = NULL;
        count = 0;
    }

    // push operation
    void push(int x) {
        Node* temp = new Node(x);
        temp->next = top;
        top = temp;

        count++;
    }

    // pop operation
    int pop() {
        if (top == NULL) {
```

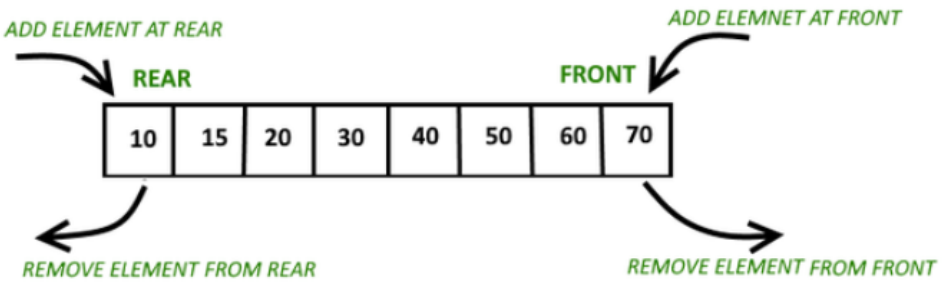
```
            cout << "Stack Underflow" << endl;
            return -1;
        }
        Node* temp = top;
        top = top->next;
        int val = temp->data;

        count--;
        delete temp;
        return val;
    }

    // peek operation
    int peek() {
        if (top == NULL) {
            cout << "Stack is Empty" << endl;
            return -1;
        }
        return top->data;
    }

    // check if stack is empty
    bool isEmpty() {
        return top == NULL;
    }

    // size of stack
    int size() {
        return count;
    }
};

int main() {
    myStack st;

    // pushing elements
```

```
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);

    // popping one element
    cout << "Popped: " << st.pop() << endl;

    // checking top element
    cout << "Top element: " << st.peek() << endl;

    // checking if stack is empty
    cout << "Is stack empty: " << (st.isEmpty() ? "Yes" : "N
o") << endl;

    // checking current size
    cout << "Current size: " << st.size() << endl;

    return 0;
}
```

💻 Output
Popped : 4
Top element : 3
Is stack empty : No
Current size : 3

Implementation using Deque

A **doubly ended queue or deque** allows insertion and deletion at both ends. In a stack, we need to do insertions and deletions at one end only. We can use either end of deque (front or back) to implement a stack,



In the below implementation, we use back (or rear) of stack to do both insertions and deletions. Please note that the time complexities of all the operations (insertions and deletions at both ends) in a deque is O(1). So the time complexity of the below implementation is O(1) only.

- In Java, the deque implementation of stack is preferred over standard Stack class, because the standard Stack class is legacy class and mainly designed for multi threaded environment.
- In Python, there is no standard stack class, so we can either use list or deque. The deque implementation is preferred because it is optimized for insertion and deletion at ends.

```
#include <iostream>
#include <deque>

using namespace std;

int main() {
    deque<int> stack;

    stack.push_back(10);
    stack.push_back(20);
    stack.push_back(30);

    cout << stack.back() << " popped from deque" << endl;
    stack.pop_back();
    cout << "Top element is: " << stack.back() << endl;
```

```
    return 0;
}
```

```
30 popped from deque
Top element is : 20
```